There are a few problems with the previous approach when applied to a cloud native system. In the rest of the chapter, we'll analyze those problems, identify viable solutions for cloud native applications, and use them on top of what we have just implemented.

11.2 Managing user accounts with Keycloak

In the previous section, we added user authentication to Edge Service based on a login form. You tried logging in via a user account that was auto-generated in memory at startup time. That's fine for experimenting with Spring Security the first time, but it's not something you'll want to do in production.

As a minimum requirement, we need persistent storage for user accounts and an option to register new users. There should be a particular focus on storing passwords using a robust encryption algorithm and preventing unauthorized access to the database. Given the criticality of such a feature, it makes sense to delegate it to a dedicated application.

Keycloak (www.keycloak.org) is an open source identity and access management solution developed and maintained by the Red Hat community. It offers a broad set of features, including single sign-on (SSO), social login, user federation, multi-factor authentication, and centralized user management. Keycloak relies on standards like OAuth2, OpenID Connect, and SAML 2.0. For now, we'll use Keycloak to manage user accounts in Polar Bookshop. Later I'll show you how to use its OpenID Connect and OAuth2 features.

NOTE Spring Security provides all the necessary features to implement a user management service. If you'd like to learn more about this subject, you can refer to chapters 3 and 4 of *Spring Security in Action* by Laurențiu Spilcă (Manning, 2020).